

Relatório Técnico: Implementação e Análise de Desempenho - Max-Heap

Nome: Gabriel Vinícius da Cruz

Data: 18/06/2026

1. Objetivo do Projeto

O presente projeto tem como objetivo principal projetar, implementar e validar uma estrutura de dados de Fila de Prioridade (Priority Queue) construída sobre um modelo de árvore **Max-Heap** estritamente binária na linguagem C.

Adicionalmente, o projeto visa realizar uma análise empírica de desempenho (profiling) da operação de inserção e reordenação ascendente (algoritmo de `siftUp`), comparando os custos computacionais teóricos com o comportamento da estrutura sob carga de dados reais e pseudo aleatórias extraídas de um arquivo CSV.

2. Dependências e Requisitos

Os requisitos são:

- **Linguagem:** C (padrão C99 ou superior).
- **Compilador:** GCC (GNU Compiler Collection).
- **Sistema Operacional:** Linux, macOS ou Windows via WSL (Windows Subsystem for Linux).
- **Ferramentas Complementares:** Bash (para orquestração da compilação via scripts `.sh`).
- **Massa de Dados:** Arquivos em formato `.csv` compatíveis com os campos da estrutura interna (Nome, Matrícula, Ranking e Curso).

3. Como Executar

O repositório está equipado com scripts de automação (Shell Script) que isolam o processo de compilação em um diretório de saída temporário (`out/`), mantendo a raiz do projeto limpa.

O arquivo [README.md](#) possui especificações quanto a definição de permissão para os arquivos de shell

- **Execução Padrão:** Navegue até o diretório raiz via terminal e execute `./main.sh`. Este script compila as bibliotecas em `src/` em conjunto com `main.c` e lança a aplicação base.
- **Execução de Performance:** Para iniciar a bateria de testes de estresse do algoritmo, execute `./performance.sh`. Este script compila a árvore conectada ao arquivo `performance.c`, executando cargas dinâmicas variando de 500 a 9.000 registros contínuos, com controle de vazamentos de memória e de ponteiros (*heap allocation*).

4. Conclusão sobre a estrutura de Heap Priority Queue

A validação da estrutura Max-Heap permitiu contrastar diretamente a literatura acadêmica com o comportamento empírico do algoritmo em tempo de execução, revelando características cruciais sobre sua escalabilidade.

Complexidade Teórica (Pior Caso)

A literatura define que a complexidade de tempo para a inserção em uma Fila de Prioridade baseada em Heap é $O(\log N)$. Este cenário de *pior caso* se manifesta estritamente quando os elementos são inseridos em ordem crescente. Ao inserir o maior elemento absoluto no final do vetor base, o algoritmo de `siftUp` é obrigado a realizar permutações sucessivas, promovendo o nó através de todos os níveis de profundidade da árvore até que ele alcance a raiz. Em uma base de 9.000 registros, o limite de operações giraria em torno de aproximadamente 13 trocas.

Complexidade Observada (Caso Médio Empírico)

A bateria de testes executada no módulo `performance.c` aplicou inserções baseadas em um vetor embaralhado aleatoriamente (via *Fisher-Yates shuffle*). O relatório terminal demonstrou que, independente do crescimento exponencial da massa de dados (de 500 para 9.000 itens), a **média de iterações do `siftUp` estabilizou em aproximadamente 2.26**.

Justificativa do Fenômeno

Este resultado confirma que, para distribuições aleatórias, o **caso médio de inserção em um Heap aproxima-se do tempo constante $O(1)$** . Essa eficiência extrema ocorre devido à geometria inerente das árvores binárias completas: a vasta maioria da população reside nas últimas camadas (folhas). Quando um elemento aleatório entra na base da estrutura, a probabilidade estatística de que este seja superior aos seus pais (que já representam os maiores valores das subárvores locais) é minúscula. Portanto, a propagação cessa quase instantaneamente, exigindo raramente mais do que duas ou três comparações.

A implementação demonstrou-se, portanto, altamente escalável, gerenciando a memória dinamicamente de forma segura e garantindo eficiência de tempo contínua sob regime de dados não classificados.

